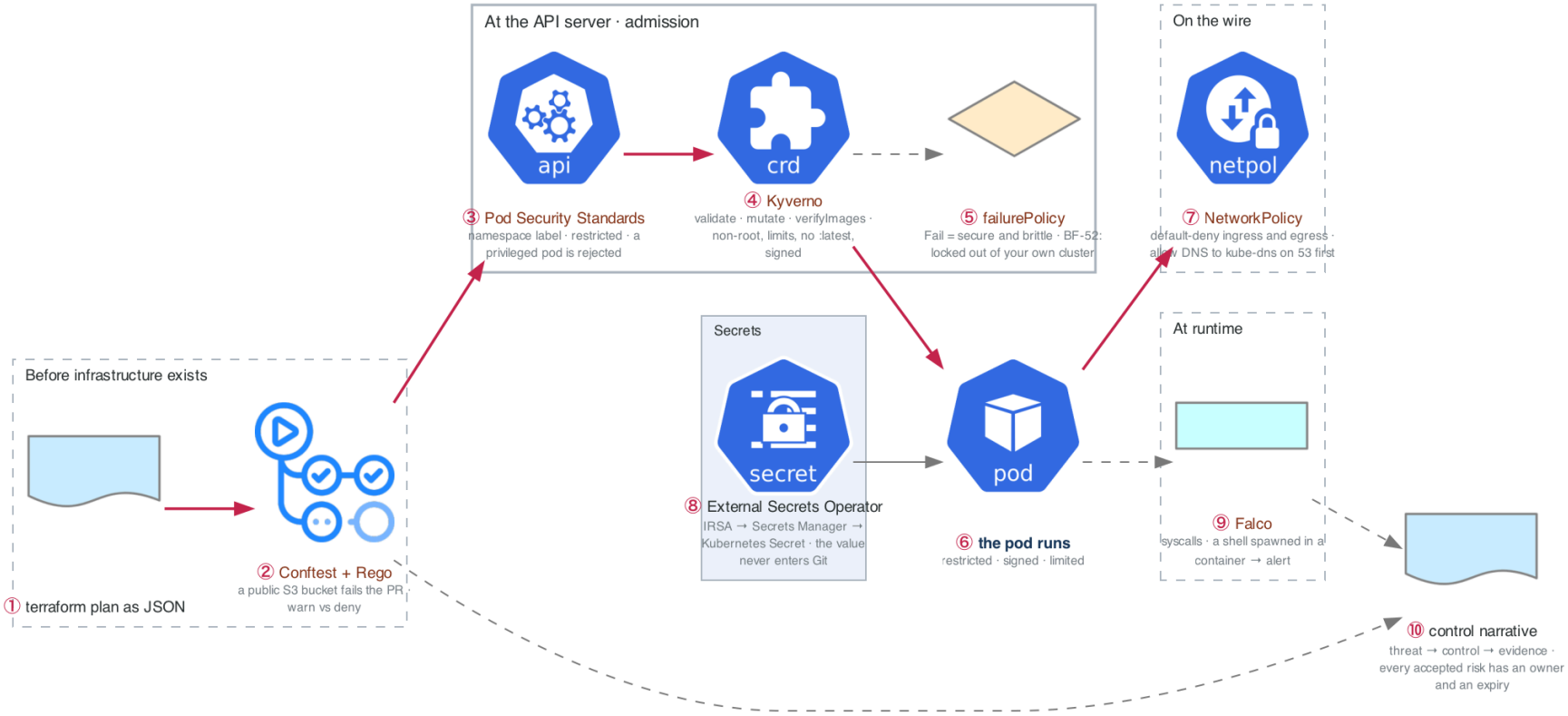
Where each control sits: before the plan, at the door, on the wire, at runtime, numbered

Week 19 · Figure 19.1 · figure with badges 1–10 · 2026-10-17



What to say, badge by badge

About four minutes at a normal pace (443 words). Point at each badge as you say its line.

▶ **Open**
“The shift this week is from 'we detect insecure deployments' to 'insecure deployments cannot happen'. This figure is a map of where the word cannot gets enforced: before the plan, at the API server, on the wire, and at runtime. Walk it left to right.”

1 **terraform plan as JSON**
“The plan is data. `terraform show -json` turns it into something a policy engine can read before anything is created.”

2 **Conftest and Rego**
“A Rego rule reads the plan and denies a public S3 bucket. The PR fails before apply. Deny fails the build; warn only shows, which is how you introduce a rule without breaking everyone on day one. Compare it with checkov: Conftest is your rules, checkov is theirs.”

3 **Pod Security Standards**
“Built in. Label a namespace `restricted` and the API server rejects a privileged pod, a root user, a host path. No controller to install. Start here, then add Kyverno for what PSS cannot express.”

4 **Kyverno**
“Policies as Kubernetes objects. Validate rejects pods without limits or with the `:latest` tag. Mutate adds defaults. verifyImages is the week-18 signature check at the door. Four rules, and all four are exit criteria.”

5 **failurePolicy**
“When the webhook is down, Fail means nothing gets admitted, Ignore means everything does. Fail is secure and brittle. BF-52 is you writing a policy that locks you out of your own cluster, then recovering by editing the webhook configuration. That is blog post 10.”

6 **the pod runs**
“Only now. It is non-root, limited, signed, in a restricted namespace. Everything above was decided before a single byte of it executed.”

7 **NetworkPolicy**
“Default-deny for ingress and egress in the namespace, then allow what the app needs. The trap every first attempt hits: without an explicit allow to kube-dns on port 53, nothing can resolve a name and everything looks broken. Demonstrate one blocked flow.”

8 **External Secrets Operator**
“The operator uses IRSA to read Secrets Manager and writes a Kubernetes Secret. The value never enters Git, never enters a CI log, and rotates when the source rotates. Bravo's secrets all arrive this way.”

9 **Falco**
“Watches syscalls on the node. A shell spawned inside a container, a write to /etc, an outbound connection from a process that never makes one: each becomes an alert. Prevention above, detection here, because prevention is never complete.”

10 **control narrative**
“A table: the threat from the STRIDE model, the control that addresses it, and the evidence that proves the control works. Accepted risks get an owner and an expiry. This is the document a security reviewer actually reads, and it is a Bravo deliverable.”

■ **Close**
“Deny in the plan, refuse at the door, drop on the wire, alert at runtime, and a table that maps each one to a threat. Prevent where you can; detect where you cannot; write both down.”